Python for Finance

Companion Slides to the Practical Sessions (TD 1–6)

M1 Economie – Finance, Université Paris Dauphine

Matéo Molinaro

2026

How to Use These Slides

These slides are a **companion** to the Jupyter notebooks used in the practical sessions (TD). They give the **full derivations** behind each practical session, self-contained, so that the slide deck can be read on its own. They point back to the notebook exercise that implements each result – but the derivations themselves do not depend on the code.

Companion sessions

TD1: Python foundations, Black–Scholes Greeks, Monte Carlo • TD2: Gradient descent, logistic MLE • TD3: Time series (AR/ARIMA/GARCH), OLS • TD4: VaR, backtesting, CoVaR • TD5: Markowitz optimization • TD6: Index construction

Session Map

Session	Topic	Notebook
TD 1	Python foundations, BSM Greeks, Monte Carlo	*Exercices - M1_eco_fi.ipynb*
TD 2	Gradient descent, logistic MLE	*Gradient Descent Students.ipynb*
TD 3	Time series: AR/ARIMA/GARCH, OLS	*Session 3 Time series modelling.ipynb*
TD 4	VaR, backtesting, CoVaR	*Session 4 Systemic risk ... .ipynb*
TD 5	Markowitz optimization	*TP Markowitz CAC 40.ipynb*
TD 6	Index construction	*Index_construction_introduction.ipynb*

Roadmap

1. TD1 – Python Foundations, BSM Greeks & Monte Carlo

Outline

1. TD1 – Python Foundations, BSM Greeks & Monte Carlo

What Is a High-Level Programming Language?

Python provides a high level of **abstraction** from the physical operation of the computer: the programmer expresses **what** problem to solve, not **how** the hardware executes it.

Python → C / C++ → Assembly → Machine instructions → CPU / Memory

Higher level / more abstraction ⟶ Lower level / closer to hardware

Key idea

A high-level language abstracts away hardware details so the programmer can express the logic of the problem directly. **High-level does not mean more advanced** – it refers only to the level of abstraction from the hardware.

Compiled vs. Interpreted Languages

Compiled (C, C++):

Source → Compiler → Machine code → CPU

The compiler analyzes and optimizes the program *in advance*; the resulting machine code runs directly – hence the high performance of compiled languages.

Interpreted (Python, in the traditional model):

Source → Interpreter → Execution

Python is commonly described this way, but this is a simplification.

What Actually Happens with Python?

CPython (the standard implementation) first **compiles** source code into an intermediate representation called **bytecode**, which is then **executed** by the Python Virtual Machine:

`.py source` → Compilation → Bytecode → Python VM → Machine instructions / CPU

Key idea

Python source code is **compiled into bytecode**, which is then **interpreted** by the Python VM. Python therefore involves both compilation and interpretation.

Programming Paradigms

A **programming paradigm** is a general approach to structuring program logic. Python is **multi-paradigm**: it supports three major paradigms, not mutually exclusive.

Paradigm	**Central idea**	**Typical features**
Imperative	Execute instructions that progressively modify program **state**	assignments, `for`, `while`, `if`
Object-Oriented	Organize **data and behavior** into interacting objects	classes, objects, attributes, methods
Functional	Express computation as **functions and transformations**, limiting state changes	`lambda`, `map`, `filter`, comprehensions

The Three Paradigms: A Simplified Distinction

One question per paradigm

Imperative: "Which instructions should the computer execute?"
Object-oriented: "Which objects should represent the system?"
Functional: "Which functions transform the inputs into outputs?"

- An **imperative** program modifies state step by step (e.g., building a list of returns inside a loop).
- An **object-oriented** program bundles **state** (attributes) and **behavior** (methods) into objects created from classes.
- A **functional** program favors **pure functions**: for the same inputs, a pure function always returns the same output with no side effects.

A realistic finance application typically **combines all three**: objects for portfolios/strategies, imperative code to step through time, functional transformations to compute signals.

Sets: Definition and Creation

A **set** is a built-in collection of **unique** elements, useful for removing duplicates, testing membership, and performing mathematical set operations.

Important detail

An empty set must be created with `set()`, **not** `{}` – the latter creates an empty **dictionary**.

Property	**List**	**Set**
Syntax	`[a,b,c]`	`{a,b,c}`
Duplicates	Allowed	Not allowed
Positional order	Yes	No
Indexing	Yes	No
Membership test	$O(n)$	$O(1)$ average

A set should be viewed as a collection based on **membership**, not position.

Why Is Set Membership $O(1)$?

Sets (and dictionaries) are implemented using **hash tables**: Python computes a numeric hash of each element and uses it to locate the element directly, without scanning the collection.

membership test on a set $= O(1)$ on average

membership test on a list $= O(n)$ worst case

Concrete example

`universe = {"AAPL", "MSFT", "GOOG"}`. To evaluate `"MSFT" in universe`, Python computes `hash("MSFT")`, a number that points **directly** to the bucket (memory slot) where `"MSFT"` is stored – one direct jump, regardless of how many tickers the set holds. With a list, the same test would compare `"MSFT"` against `"AAPL"`, then `"MSFT"`, ..., one by one, until a match is found.

Mathematical Set Operations

For two investment universes A and B:

$\boxed{A \cup B}$ (union: in either) $\boxed{A \cap B}$ (intersection: in both)

$\boxed{A \setminus B}$ (difference: in A but not B) $\boxed{A \triangle B}$ (symmetric difference: in exactly one)

Operation	**Set notation**	**Membership**
Union	$A \cup B$	$x \in A$ or $x \in B$
Intersection	$A \cap B$	$x \in A$ and $x \in B$
Difference	$A \setminus B$	$x \in A$ and $x \notin B$
Symmetric difference	$A \triangle B$	x in exactly one of A, B

Hashability: What Does "Hashable" Mean?

An object is **hashable** if Python can associate it with a **hash value** that remains stable throughout the object's lifetime:

$$\boxed{h : \text{Object} \longrightarrow \mathbb{Z}}$$

Python uses hash values internally to locate objects efficiently inside **hash tables** – the data structure underlying both `dict` and `set`. This is exactly why membership testing in a set or dictionary is $O(1)$ on average: integers, strings, and tuples of hashable objects are generally hashable.

A Concrete Example: Building a Hash Table

Consider building `prices` as a hash table with 8 slots, by inserting three key–value pairs:

```
prices["AAPL"]=150;  prices["MSFT"]=310;  prices["GOOG"]=2800
```

1. `hash("AAPL")` $= 41 \Rightarrow 41 \bmod 8 = 1$: store `("AAPL",150)` in slot 1.
2. `hash("MSFT")` $= 26 \Rightarrow 26 \bmod 8 = 2$: store `("MSFT",310)` in slot 2.
3. `hash("GOOG")` $= 58 \Rightarrow 58 \bmod 8 = 2$: slot 2 is already taken (**collision**) $\Rightarrow$ probe the next slot, slot 3: store `("GOOG",2800)` in slot 3.

Slot 0	Slot 1	Slot 2	Slot 3	Slot 4	Slot 5	Slot 6	Slot 7
–	AAPL:150	MSFT:310	GOOG:2800	–	–	–	–

The numbers above are illustrative: real hash values are large (and, for strings, randomized across runs); only their remainder modulo the table size matters here.

Looking Up a Key: Why It Is Direct

To evaluate `prices["MSFT"]`, Python repeats the **same** computation used at insertion time:

1. Compute `hash("MSFT")` $= 26$.
2. Compute the slot index: $26 \bmod 8 = 2$.
3. Jump **directly** to slot 2, compare the stored key `"MSFT"` to the requested key: match $\Rightarrow$ return `310` immediately.

slots $0, 1, 3, 4, \ldots$ are never even visited: one hash computation + one direct jump

What about collisions?

If the slot reached by hashing holds a *different* key (as with `"GOOG"` above), Python probes further slots until it finds the matching key (lookup) or an empty slot (insertion). This is why membership testing is $O(1)$ **on average**, not in every single case.

Why Is a List Not Hashable?

A `list` is **mutable**: its contents can change after creation. If lists were hashable, mutating a list would change the very information its hash is computed from:

$$[1,2,3] \xrightarrow{\text{hash}} h_1 \qquad \text{but, after appending 4,} \qquad [1,2,3,4] \xrightarrow{\text{hash}} h_2 \neq h_1$$

Consequence

This would violate the requirement that a hash-table key's hash stays **stable**. Python therefore deliberately makes lists **unhashable** – so a list can never be an element of a `set` or a key of a `dict`.

List vs. Tuple, and a Summary of Hashability

A `list` is mutable ⇒ not hashable. A `tuple` is **immutable** and, provided all its elements are themselves hashable, **is** hashable – so a tuple (e.g. `(ticker, date)`) can be used as a dictionary key.

Type	Mutable?	Hashable?
`int`, `float`, `str`	No	Yes
`tuple`	No	Usually yes*
`list`, `set`	Yes	No

*A tuple is hashable only if all of its elements are hashable.

Key idea

A hashable object has a stable hash and can be used in hash-based structures (set elements, dict keys). A list is mutable, so Python deliberately makes it unhashable.

The `for` Loop: Iterating over Lists, Ranges, Dictionaries

A `for` loop successively assigns each element of a collection to a loop variable:

$$\texttt{stock} = \texttt{AMZN} \to \texttt{TSLA} \to \texttt{AAPL} \to \texttt{META}$$

Ranges exclude their upper bound: `range(1,10)` generates $\{1, 2, \ldots, 9\}$ – **10 is excluded**.

Iterating over a dictionary

Pairing two lists with `zip()` associates same-position elements:
$\texttt{AMZN} \leftrightarrow 133.62$, $\texttt{TSLA} \leftrightarrow 889.36, \ldots$ Calling `.items()` on the resulting dictionary then yields each $(\text{key}, \text{value})$ pair, unpacked at every iteration into two loop variables.

The `while` Loop and the Role of `break`

A `while` loop executes as long as its condition is `True`. Using `while True:` creates an **infinite loop**, since the condition can never become `False` on its own:

$$\texttt{True} \rightarrow \text{execute} \rightarrow \texttt{True} \rightarrow \text{execute} \rightarrow \cdots$$

The loop must therefore be terminated **explicitly** with `break`, triggered by a stopping condition evaluated at each pass.

Boolean convention for lists

$$\texttt{bool(non-empty list)} = \texttt{True} \qquad \texttt{bool(empty list)} = \texttt{False}$$

So once a list being emptied becomes `[]`, the condition on it flips to `False` and `break` fires.

Worked Example: Transferring Elements Between Two Lists

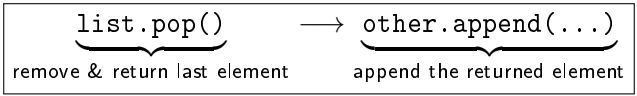
Transferring the last element of `portfolio2` into `portfolio1`, repeatedly, until `portfolio2` is empty:

Iteration	Element transferred	portfolio2 after
1	`F`	[V, TTE, FTKE]
2	`FTKE`	[V, TTE]
3	`TTE`	[V]
4	`V`	[]

Because the transfer always takes the **last** element (index -1), elements end up **reversed** in `portfolio1`: the final list is `[AMZN, TSLA, AAPL, META, F, FTKE, TTE, V]`.

Why pop() Lets Us Write One Instruction Instead of Two

The method `pop()` does **two things at once**: it removes the last element from a list **and** returns it.



This means two separate instructions – access the last element, then remove it – collapse into a **single** instruction that appends the value *returned by* the removal itself. Both forms have the **same** asymptotic complexity, $O(n)$ for transferring n elements (`pop()`/`append()` are each $O(1)$ amortized): the gain from combining them is **conciseness**, not speed.

Idiomatic Simplification of the Stopping Condition

Because Python treats an empty list as `False`, the explicit test `not bool(list)` can be shortened:

```
if not bool(portfolio2):  break   ⟹   if not portfolio2:  break
```

And the condition can be moved directly into the loop header, which reads naturally as "continue while `portfolio2` is not empty":

```
while portfolio2:  ...
```

This is the most concise and idiomatic version of the pattern: no explicit `break` is needed at all, because the loop's own condition already encodes the stopping rule.

The Membership Operator `in`

The instruction `if x in collection:` tests whether `x` belongs to `collection`, using the **membership operator**:

$$\texttt{x in collection} = \begin{cases} \texttt{True}, & \text{if } x \text{ belongs to the collection} \\ \texttt{False}, & \text{otherwise} \end{cases}$$

In an `if/else` statement built on this test, the `if`-branch executes when the condition is `True`, the `else`-branch when it is `False` – e.g., print a message if a stock is already held, otherwise append it to the holdings.

Step-by-Step Trace of a Membership-Based Loop

Scanning a candidate list `stocks2 = [V, TTE, AAPL, MS, AMZN]` against a growing `my_stocks`:

Stock	Already present?	Condition	Action
V	No	False	Append V
TTE	No	False	Append TTE
AAPL	Yes	True	Print message
MS	No	False	Append MS
AMZN	Yes	True	Print message

$$[\text{AMZN},\text{TSLA},\text{AAPL},\text{META}] \xrightarrow{+V} \xrightarrow{+TTE} \xrightarrow{+MS} [\text{AMZN},\text{TSLA},\text{AAPL},\text{META},\text{V},\text{TTE},\text{MS}]$$

Important Detail: the Condition Is Evaluated Against a Mutating List

The membership test `stocks in my_stocks` is re-evaluated against `my_stocks` **as it stands at that point in the loop** – and `my_stocks` is itself being modified inside the loop.

Consequence

After the first append, `my_stocks` already contains the newly added element, so **all subsequent** membership tests run against this **updated** list, not the original one. Any element added earlier in the same pass is immediately visible to later iterations.

Key idea

An `if` statement checks a condition: the `if`-block runs when it is `True`, the `else`-block when it is `False` – and when that condition reads a mutable collection, its truth can change **during** the very loop that is testing it.

Referencing a Method vs. Calling It

Writing `obj.method` **without** parentheses does **not** execute it: Python simply stores a **reference** to the method object.

$$\boxed{\texttt{method_open} \longrightarrow \texttt{my_car.open_door}} \qquad (\texttt{type(method_open)} = \texttt{method})$$

Adding parentheses **executes** it and returns its result:

$$\boxed{\texttt{method_open} \xrightarrow{()} \text{execute} \xrightarrow{\texttt{return}} \text{the returned value}}$$

Method vs. Return Value: Two Different Types

$$\underbrace{\texttt{method_open}}_{\text{the method itself, type } \texttt{method}} \neq \underbrace{\texttt{method_open()}}_{\text{result of executing it, e.g. type } \texttt{str}}$$

Storing the method (`m = obj.method`) lets you call it repeatedly later: each `m()` re-executes it. **Storing the result** (`r = obj.method()`) captures a fixed value computed **once**, at call time – `r` is then just that value (e.g. a string), not a callable.

Key idea

`obj.method` refers to the method itself; `obj.method()` calls it and returns its result. In Python, functions and methods are themselves objects: they can be assigned to variables and invoked later.

What Happens to `self`?

A method defined as `def open_door(self):` is called as `my_car.open_door()` – with **no** explicit argument – because Python **automatically** passes the instance as the first parameter:

`my_car.open_door()` is equivalent to `Voiture.open_door(my_car)`

$$\boxed{\texttt{self} = \texttt{my_car}}$$

This is exactly what lets a method reach into the attributes of **the specific instance** it was called on (e.g., `self.marque`, `self.couleur`) – the same method code produces instance-specific behavior.

Reading HTML Tables: a List of DataFrames, Not One

Reading an HTML page for tables does **not** return a single table – it returns a **list** of tables, one per table detected on the page:

$$\boxed{\text{HTML-table reader}(\textit{url}) \longrightarrow [\text{DataFrame}_0, \text{DataFrame}_1, \ldots, \text{DataFrame}_n]}$$

The desired table must therefore be **selected by index** from that list before it can be used as a normal DataFrame.

First Inspection: Never Assume Scraped Data Is Analysis-Ready

Before any analysis, inspect structure first:

Diagnostic	Purpose
first rows	inspect the first observations
shape	number of rows and columns
columns / index	names of columns / row index
dtypes	data type of each column
general-info summary	overall structure and missing values

Five Problems to Check in Raw Scraped Data

1. **Missing values** – may need deletion, imputation, or another treatment.
2. **Incorrect data types** – a numeric column may be imported as text if it contains symbols, footnotes, or stray characters, and must be converted.
3. **Unnecessary columns** – irrelevant columns should be dropped.
4. **Column names** – names may need simplifying/standardizing.
5. **Duplicates** – duplicated rows should be detected and, if appropriate, removed.

From Raw HTML Data to Usable Data: the Pipeline

HTML page → table extraction → raw DataFrame → inspection → cleaning → analysis

Key idea

Importing data is only the first step. External data should always be **inspected, validated, and cleaned** before being used for statistical or financial analysis.

The Lookup Dictionary as a Mapping

A replacement dictionary such as {`Nvidia:AMD`, `BoFa:Morgan Stanley`} is, mathematically, a **mapping**:

$$\texttt{Nvidia} \longrightarrow \texttt{AMD}, \qquad \texttt{BoFa} \longrightarrow \texttt{Morgan Stanley}$$

Keys are the words to detect; values are their replacements. Any word that is **not** a key must remain unchanged – this is the whole design goal of the algorithm that follows.

Splitting the String, and a Punctuation Pitfall

Splitting a sentence on whitespace yields a list of tokens, but splitting **does not remove punctuation**: a token immediately followed by a comma in the sentence keeps that comma attached (e.g. `Nvidia,`), which would make a direct dictionary lookup on the raw token fail even though the underlying word is a key.

Detecting and Temporarily Stripping the Comma

A conditional expression `value_if_true if condition else value_if_false` decides, per token, whether a comma was present by comparing the token to its comma-stripped version:

$$\texttt{comma} = \begin{cases} \texttt{False} & \text{if stripping commas changes nothing} \\ \texttt{True} & \text{if stripping commas changes the token} \end{cases}$$

token	comma-stripped	comma?
`Ford`	`Ford`	False
`Apple,`	`Apple`	True
`Nvidia,`	`Nvidia`	True

Dictionary Lookup with a Default: `get(key, default)`

A dictionary lookup with a fallback value returns the mapped value **if** the key exists, and the fallback **otherwise**:

$$\texttt{dict.get(key, default)} = \begin{cases} \text{value mapped to } \texttt{key}, & \text{if } \texttt{key} \text{ is present} \\ \texttt{default}, & \text{otherwise} \end{cases}$$

Passing the token **itself** as the default is exactly what lets unmapped words pass through **unchanged** – no special-casing is needed for words outside the lookup dictionary.

Three Cases of the Replace-and-Restore Logic

Each token is processed by: strip comma if present → look up → restore comma if it had one.

`"Nvidia,"` → `"Nvidia"` → `"AMD"` → `"AMD,"` (comma, in dictionary)

`"BoFa"` → `"Morgan Stanley"` (no comma, in dictionary)

`"Apple,"` → `"Apple"` → `"Apple"` → `"Apple,"` (comma, **not** in dictionary: unchanged)

Every other token (`I`, `have`, `Ford`, `and`, `in`, `my`, `portfolio`) is not in the dictionary and passes through unchanged.

Reassembling the String and the Full Pipeline

Joining the processed tokens with a single space between them reconstructs one string from the list:

$$\boxed{\text{list of strings} \xrightarrow{\text{join with spaces}} \text{single string}}$$

$$\boxed{\begin{array}{c}\text{Original string} \to \text{split into words} \to \text{for each word:} \\ \text{detect \& strip comma} \to \text{look up in dictionary (default = itself)} \to \text{restore comma} \\ \to \text{collect into a list} \to \text{join with spaces} \to \text{reconstructed string}\end{array}}$$

Key idea

A dictionary used as a lookup table replaces exactly the words it knows about and leaves everything else – including punctuation – unchanged.

Euclidean Distance: Mathematical Definition

For two points $P_1 = (x_1, \ldots, x_n)$ and $P_2 = (y_1, \ldots, y_n)$ in $\mathbb{R}^n$, the Euclidean distance is the norm of their difference vector:

$$d(P_1, P_2) = \|P_1 - P_2\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Example: $P_1 = (3, 4)$, $P_2 = (0, 0) \Rightarrow d = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$.

Default argument semantics

A distance function with a default second point equal to the origin $(0, \ldots, 0)$ computes, by default, the distance of a single point **to the origin** – i.e. its norm $\|P_1\|_2$.

The Pipeline, Step by Step

The computation of $d(P_1, P_2)$ decomposes into five elementary steps:

$$[x_i - y_i]_i \xrightarrow{\text{square each}} [(x_i - y_i)^2]_i \xrightarrow{\text{sum}} \sum_i (x_i - y_i)^2 \xrightarrow{\sqrt{\cdot}} d(P_1, P_2)$$

1. Build the component-wise **difference vector** $[x_i - y_i]_i$.
2. **Square** every component: $x \mapsto x^2$.
3. **Sum** the squared components: $\sum_i (x_i - y_i)^2$.
4. Take the **square root** of the sum.

A nested expression such as $\sqrt{\sum(\text{ squares of}(\text{ difference }))}$ must be read **from the inside out**: difference $\to$ square $\to$ sum $\to$ square root.

Worked Example and an Implicit Assumption

For $P_1 = (3,4)$, $P_2 = (0,0)$:

$$[3,4] \xrightarrow{\text{subtract}} [3,4] \xrightarrow{\text{square}} [9,16] \xrightarrow{\text{sum}} 25 \xrightarrow{\sqrt{\cdot}} 5$$

Hidden assumption: equal dimension

$$\boxed{\text{len}(P_1) = \text{len}(P_2)}$$

If the two points do not have the same number of coordinates, the component-wise difference is not defined: the computation fails as soon as it tries to access a coordinate that does not exist. A default point such as $(0,0)$ silently assumes $\mathbb{R}^2$.

Rows of a Matrix as Points

Each row of a matrix M can be read as a point in $\mathbb{R}^n$. Given a reference vector V, ranking the rows by their distance to V means comparing

$$d(P_1, V), \quad d(P_2, V), \quad \ldots, \quad d(P_k, V)$$

for every row P_i of M, then ordering the rows by these values.

Sorting by a Computed Key

A generic sort-by-key routine orders the elements of a collection according to the value returned by a **key function** applied to each element – it does not change what the elements are, only the order in which they come out. Using

$$\text{key}(x) = d(x, V) = \|x - V\|_2$$

as the sorting key means: *"sort the rows of M according to their distance from V."* An anonymous (lambda) key function computing $d(x, V)$ for a fixed V is exactly equivalent to a small named function doing the same thing.

Worked Example: Three Points, One Reference Vector

For $M = \begin{pmatrix} 3 & 4 \\ 1 & 2 \\ 5 & 5 \end{pmatrix}$ and $V = (1, 1)$:

$$d((3,4), V) = \sqrt{2^2 + 3^2} = \sqrt{13} \approx 3.61$$
$$d((1,2), V) = \sqrt{0^2 + 1^2} = 1$$
$$d((5,5), V) = \sqrt{4^2 + 4^2} = \sqrt{32} \approx 5.66$$

Since $1 < 3.61 < 5.66$, sorting by distance (ascending, the default) orders the rows as

$$\boxed{(1,2),\ (3,4),\ (5,5)}$$

From "Sort Everything" to "Find the Closest Point"

Sorting all rows by distance and taking the first element answers a more specific question than was asked: we only need the **single closest** point, i.e.

$$x^* = \arg\min_{x \in M} \|x - V\|_2$$

A direct minimum is cheaper than a full sort

Sorting all n rows costs $O(n \log n)$; scanning once for the minimum costs only $O(n)$, since each row needs to be examined only once. A **min-with-key** routine is therefore both more explicit (it directly expresses "smallest distance") and asymptotically cheaper when only the closest point is needed.

Key idea

The sorting/minimizing key does not change which elements exist – it tells the comparison **which computed value** to compare elements by.

Magic Squares: Definition

A **magic square** of order n is an $n \times n$ array of n^2 strictly positive integers

$$M = \begin{pmatrix} m_{11} & m_{12} & \cdots & m_{1n} \\ m_{21} & m_{22} & \cdots & m_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & \cdots & m_{nn} \end{pmatrix}$$

such that every row, every column, and both main diagonals sum to the **same** value – the **magic constant** S.

The Four Conditions a Magic Square Must Satisfy

Row sums and column sums:

$$R_i = \sum_{j=1}^{n} m_{ij}, \qquad C_j = \sum_{i=1}^{n} m_{ij}$$

Main and secondary diagonal sums:

$$D_1 = \sum_{i=1}^{n} m_{ii}, \qquad D_2 = \sum_{i=1}^{n} m_{i,\,n+1-i}$$

$$\boxed{M \text{ is magic} \iff R_1 = \cdots = R_n = C_1 = \cdots = C_n = D_1 = D_2 = S}$$

Every row sum, every column sum, and both diagonal sums must agree on the **exact same** number S.

Worked Example: Is This 3×3 Square Magic?

$$M = \begin{pmatrix} 8 & 1 & 6 \\ 3 & 5 & 7 \\ 4 & 9 & 2 \end{pmatrix}$$

Rows	Columns	Diagonals
$8+1+6=15$	$8+3+4=15$	$8+5+2=15$
$3+5+7=15$	$1+5+9=15$	$6+5+4=15$
$4+9+2=15$	$6+7+2=15$	

$\boxed{R_1 = R_2 = R_3 = C_1 = C_2 = C_3 = D_1 = D_2 = 15}$ $\Longrightarrow$ this **is** a magic square.

From Math to a Single Pass Over the Square

With Python's zero-based indexing, `square[i][j]` plays the role of m_{ij}. Two observations let every condition be checked in **one** pass over the array:

1. At fixed i, looping j from 0 to $n-1$ gives **both** the row-i term `square[i][j]` **and** the column-i term `square[j][i]` – so row i and column i can be summed in the **same** inner loop.
2. At that same index i, `square[i][i]` is the main-diagonal term and `square[i][n-1-i]` is the secondary-diagonal term – both diagonals can be accumulated **inside the outer loop**, one term per iteration.

Algorithm

Take `magic_sum = sum(square[0])` as the target S; for each i, accumulate the row-i and column-i sums (returning `False` immediately on a mismatch) and add one term to each diagonal sum; after the loop, compare both diagonal sums to `magic_sum`.

$$\text{cost} = O(n^2): \text{ every entry of the } n \times n \text{ square is visited exactly once}$$

`startswith()`: Definition and Case Sensitivity

Testing whether a string begins with a given prefix returns a Boolean:

$$\texttt{string.startswith(prefix)} = \begin{cases} \texttt{True}, & \text{the string begins with } \texttt{prefix} \\ \texttt{False}, & \text{otherwise} \end{cases}$$

A string s has prefix p exactly when $s = p +$ (the remaining suffix), e.g. `"Apple"` $=$ `"App"` $+$ `"le"`.

Case sensitivity

String comparison is **case-sensitive**: `"A"` $\neq$ `"a"`, so `"Apple"` does not start with `"app"`. A case-insensitive test requires lower-casing both the string and the prefix before comparing.

Counting Words by Prefix: the General Pattern

Selecting, from a list of words, only those satisfying a condition is a **filtered list construction**:

$$[\text{element} : \text{element in collection},\ \text{condition}(\text{element})]$$

With `condition(word) = word.startswith(prefix)` on the tokens of a sentence:

word	starts with "A"?	selected?
Apple	True	Yes
Amazon	True	Yes
Microsoft	False	No
AMD	True	Yes

Counting the elements of the filtered collection answers "how many words start with this prefix?".

The Full Pipeline, and a Boolean Shortcut

sentence → split into words → keep words with `startswith(prefix)` True → count them

`"Apple Amazon Microsoft AMD"` $\xrightarrow{\text{split}}$ [...] $\xrightarrow{\text{startswith("A")}}$ [Apple, Amazon, AMD] $\xrightarrow{\text{count}}$ 3

A shortcut via Boolean arithmetic

In Python, `True` $= 1$ and `False` $= 0$, so **summing** a sequence of Boolean test results directly counts how many are `True` – no intermediate filtered list is even needed: $1 + 1 + 0 + 1 = 3$.

append() vs. extend()

Adding a list as **one** element nests it; adding **its elements individually** flattens them in:

$$[1, 2] \xrightarrow{\texttt{append([3,4])}} [1, 2, [3, 4]]$$

$$[1, 2] \xrightarrow{\texttt{extend([3,4])}} [1, 2, 3, 4]$$

`append()` = add one object

`extend()` = add all elements of an iterable

Why `extend()` Is Needed to Pool Observations Across Groups

Given several stocks, each with its own list of monthly returns, accumulating **all individual return observations** into one flat list (so that, e.g., a single median can be computed across every observation) requires `extend()`, not `append()`:

$$[0.02, 0.03, -0.01] \qquad [0.05, 0.04, 0.08] \qquad [0.01, -0.02, 0.03]$$
$$\downarrow \text{ extend, one group at a time}$$
$$[0.02, 0.03, -0.01,\ 0.05, 0.04, 0.08,\ 0.01, -0.02, 0.03]$$

The `append()` trap

Using `append()` instead would nest each group's list as a **single element**, producing $[[0.02, 0.03, -0.01], [0.05, 0.04, 0.08], [0.01, -0.02, 0.03]]$ – a list of lists, not the flat list of observations needed to compute a statistic across all of them at once.

The Black–Scholes Prices

We consider the Black–Scholes prices of a European call and put:

$$C(S,t) = S\,N(d_1) - Ke^{-rT}N(d_2), \qquad P(S,t) = Ke^{-rT}N(-d_2) - S\,N(-d_1),$$

with

$$d_1 = \frac{\ln(S/K) + \left(r + \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}.$$

We denote by $N(x) = \int_{-\infty}^{x} \phi(u)\,du$ the standard normal CDF and by $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$ its density.

A fundamental identity

$$\boxed{\frac{d}{dx}N(x) = \phi(x)}$$

since the derivative of a CDF is the corresponding density.

Useful Intermediate Derivatives (w.r.t. S)

Since $d_1 = \dfrac{\ln(S/K) + \left(r + \frac{1}{2}\sigma^2\right) T}{\sigma\sqrt{T}}$, we have, with respect to S,

$$\frac{\partial d_1}{\partial S} = \frac{1}{S\sigma\sqrt{T}}.$$

Because $d_2 = d_1 - \sigma\sqrt{T}$ (a shift that does not depend on S),

$$\boxed{\frac{\partial d_2}{\partial S} = \frac{1}{S\sigma\sqrt{T}}}$$

Both d_1 and d_2 have the same sensitivity to S – this will make several Greek derivations collapse nicely.

Useful Intermediate Derivatives (w.r.t. σ)

Rewrite d_1 to isolate its σ-dependence:

$$d_1 = \frac{\ln(S/K) + rT}{\sigma\sqrt{T}} + \frac{1}{2}\sigma\sqrt{T}.$$

Differentiating,

$$\frac{\partial d_1}{\partial \sigma} = -\frac{\ln(S/K) + rT}{\sigma^2\sqrt{T}} + \frac{1}{2}\sqrt{T}.$$

Using $d_2 = d_1 - \sigma\sqrt{T}$,

$$\boxed{\frac{\partial d_2}{\partial \sigma} = \frac{\partial d_1}{\partial \sigma} - \sqrt{T}}$$

A Key Identity: $S\phi(d_1) = Ke^{-rT}\phi(d_2)$

This identity is what makes several terms cancel throughout the Greeks derivations. We have

$$\phi(d_1) = \frac{1}{\sqrt{2\pi}} e^{-d_1^2/2}, \qquad \phi(d_2) = \frac{1}{\sqrt{2\pi}} e^{-d_2^2/2} \quad \Rightarrow \quad \frac{\phi(d_1)}{\phi(d_2)} = \exp\left(-\tfrac{1}{2}d_1^2 + \tfrac{1}{2}d_2^2\right).$$

Since $d_2 = d_1 - \sigma\sqrt{T}$,

$$d_2^2 - d_1^2 = -2d_1\sigma\sqrt{T} + \sigma^2 T \quad \Rightarrow \quad \frac{\phi(d_1)}{\phi(d_2)} = \exp\left(-d_1\sigma\sqrt{T} + \tfrac{1}{2}\sigma^2 T\right).$$

A Key Identity (cont'd)

Now $d_1\sigma\sqrt{T} = \ln(S/K) + \left(r + \frac{1}{2}\sigma^2\right) T$, so

$$-d_1\sigma\sqrt{T} + \tfrac{1}{2}\sigma^2 T = -\ln(S/K) - rT.$$

Therefore

$$\frac{\phi(d_1)}{\phi(d_2)} = e^{-\ln(S/K)-rT} = \frac{K}{S}e^{-rT}.$$

Result

$$\boxed{S\phi(d_1) = Ke^{-rT}\phi(d_2)}$$

Call Delta: $\Delta = \partial V / \partial S$

For a call, $C = SN(d_1) - Ke^{-rT}N(d_2)$. Differentiating w.r.t. S and using the product rule on $SN(d_1)$:

$$\frac{\partial C}{\partial S} = \underbrace{N(d_1) + S\phi(d_1)\frac{\partial d_1}{\partial S}}_{\partial[SN(d_1)]/\partial S} - Ke^{-rT}\phi(d_2)\frac{\partial d_2}{\partial S}.$$

Substituting $\frac{\partial d_1}{\partial S} = \frac{\partial d_2}{\partial S} = \frac{1}{S\sigma\sqrt{T}}$:

$$\frac{\partial C}{\partial S} = N(d_1) + S\phi(d_1)\frac{1}{S\sigma\sqrt{T}} - Ke^{-rT}\phi(d_2)\frac{1}{S\sigma\sqrt{T}}.$$

Using the key identity $S\phi(d_1) = Ke^{-rT}\phi(d_2)$, the last two terms cancel exactly:

$$\boxed{\Delta_C = N(d_1)}$$

Put Delta via Put–Call Parity

Put–call parity gives $P = C - S + Ke^{-rT}$. Differentiating w.r.t. S:

$$\frac{\partial P}{\partial S} = \frac{\partial C}{\partial S} - 1.$$

Result

$$\boxed{\Delta_P = N(d_1) - 1}$$

Equivalently, using $N(x) - 1 = -N(-x)$: $\Delta_P = -N(-d_1)$.

Gamma: $\Gamma = \partial^2 V / \partial S^2 = \partial \Delta / \partial S$

For the call, $\Delta_C = N(d_1)$, so

$$\Gamma_C = \phi(d_1) \frac{\partial d_1}{\partial S} = \phi(d_1) \cdot \frac{1}{S\sigma\sqrt{T}}.$$

$$\boxed{\Gamma_C = \frac{\phi(d_1)}{S\sigma\sqrt{T}}}$$

For the put, $\Delta_P = N(d_1) - 1$: the derivative of the constant -1 is zero, so

$$\boxed{\Gamma_P = \Gamma_C = \frac{\phi(d_1)}{S\sigma\sqrt{T}}}$$

Calls and puts with identical parameters therefore have the **same Gamma**.

Vega: $\nu = \partial V/\partial\sigma$ (Call)

Differentiating $C = SN(d_1) - Ke^{-rT}N(d_2)$ w.r.t. σ:

$$\frac{\partial C}{\partial \sigma} = S\phi(d_1)\frac{\partial d_1}{\partial \sigma} - Ke^{-rT}\phi(d_2)\frac{\partial d_2}{\partial \sigma}.$$

Using $\dfrac{\partial d_2}{\partial \sigma} = \dfrac{\partial d_1}{\partial \sigma} - \sqrt{T}$ and expanding:

$$\frac{\partial C}{\partial \sigma} = \underbrace{\left[S\phi(d_1) - Ke^{-rT}\phi(d_2)\right]}_{=0 \text{ by the key identity}}\frac{\partial d_1}{\partial \sigma} + Ke^{-rT}\phi(d_2)\sqrt{T}.$$

The bracket vanishes, leaving $\dfrac{\partial C}{\partial \sigma} = Ke^{-rT}\phi(d_2)\sqrt{T}$; applying the identity once more:

$$\boxed{\nu_C = S\phi(d_1)\sqrt{T}}$$

Vega: Put, and the Per-Point Convention

By an identical argument starting from the put price (or directly from put–call parity, since $\partial(S - Ke^{-rT})/\partial\sigma = 0$):

$$\boxed{\nu_P = S\phi(d_1)\sqrt{T}} \qquad \Longrightarrow \qquad \boxed{\nu_C = \nu_P}$$

Per 1-point convention

If Vega is reported for a one-percentage-point move in volatility:

$$\boxed{\nu_{1\%} = \frac{S\phi(d_1)\sqrt{T}}{100}}$$

Theta: Two Conventions

Theta is the sensitivity of the option price to time. If t denotes calendar time,

$$\boxed{\Theta = \frac{\partial V}{\partial t}}$$

Since $T = T_{\text{maturity}} - t$, we have $\dfrac{\partial T}{\partial t} = -1$, hence by the chain rule

$$\boxed{\Theta = \frac{\partial V}{\partial t} = -\frac{\partial V}{\partial T}}$$

We now derive the usual *market* Theta, $\Theta = -\partial V / \partial T$.

Call Theta: Differentiating w.r.t. T

Starting from $C = SN(d_1) - Ke^{-rT}N(d_2)$ and using the product rule on the second term:

$$\frac{\partial}{\partial T}\left[Ke^{-rT}N(d_2)\right] = -rKe^{-rT}N(d_2) + Ke^{-rT}\phi(d_2)\frac{\partial d_2}{\partial T},$$

so that

$$\frac{\partial C}{\partial T} = S\phi(d_1)\frac{\partial d_1}{\partial T} + rKe^{-rT}N(d_2) - Ke^{-rT}\phi(d_2)\frac{\partial d_2}{\partial T}.$$

Using the key identity $S\phi(d_1) = Ke^{-rT}\phi(d_2)$, the first and third terms combine:

$$\frac{\partial C}{\partial T} = S\phi(d_1)\left(\frac{\partial d_1}{\partial T} - \frac{\partial d_2}{\partial T}\right) + rKe^{-rT}N(d_2).$$

Call Theta: Final Result

Since $d_2 = d_1 - \sigma\sqrt{T}$,

$$\frac{\partial d_2}{\partial T} = \frac{\partial d_1}{\partial T} - \frac{\sigma}{2\sqrt{T}} \quad \Longrightarrow \quad \frac{\partial d_1}{\partial T} - \frac{\partial d_2}{\partial T} = \frac{\sigma}{2\sqrt{T}}.$$

Hence $\dfrac{\partial C}{\partial T} = \dfrac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(d_2)$, and since market Theta is $\Theta_C = -\partial C/\partial T$:

$$\boxed{\Theta_C = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2)}$$

Put Theta via Put–Call Parity

From $P = C - S + Ke^{-rT}$, differentiating w.r.t. calendar time t:

$$\Theta_P = \Theta_C + rKe^{-rT}.$$

Substituting the call Theta and factoring the last two terms:

$$\Theta_P = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2) + rKe^{-rT} = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}\big[1 - N(d_2)\big].$$

Using $1 - N(x) = N(-x)$:

$$\boxed{\Theta_P = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(-d_2)}$$

Daily convention: $\Theta_{\text{daily}} = \Theta_{\text{annual}}/365$.

Call Rho: $\rho = \partial V / \partial r$

We need $\dfrac{\partial d_1}{\partial r} = \dfrac{T}{\sigma\sqrt{T}} = \dfrac{\sqrt{T}}{\sigma} = \dfrac{\partial d_2}{\partial r}$. Differentiating $C = SN(d_1) - Ke^{-rT}N(d_2)$ w.r.t. r:

$$\frac{\partial C}{\partial r} = S\phi(d_1)\frac{\sqrt{T}}{\sigma} + KTe^{-rT}N(d_2) - Ke^{-rT}\phi(d_2)\frac{\sqrt{T}}{\sigma}.$$

Using the key identity $S\phi(d_1) = Ke^{-rT}\phi(d_2)$, the first and third terms cancel:

$$\boxed{\rho_C = KTe^{-rT}N(d_2)}$$

Put Rho via Put–Call Parity

Differentiating $P = C - S + Ke^{-rT}$ w.r.t. r:

$$\rho_P = \rho_C - KTe^{-rT} = KTe^{-rT}\big[N(d_2) - 1\big].$$

Using $N(d_2) - 1 = -N(-d_2)$:

$$\boxed{\rho_P = -KTe^{-rT}N(-d_2)}$$

Per 1-point convention: $\rho_{1\%} = \rho/100$.

Summary Table

Greek	Call	Put
Delta	$N(d_1)$	$N(d_1) - 1$
Gamma	$\frac{\phi(d_1)}{S\sigma\sqrt{T}}$	$\frac{\phi(d_1)}{S\sigma\sqrt{T}}$
Vega	$S\phi(d_1)\sqrt{T}$	$S\phi(d_1)\sqrt{T}$
Theta	$-\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2)$	$-\frac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(-d_2)$
Rho	$KTe^{-rT}N(d_2)$	$-KTe^{-rT}N(-d_2)$

Economic Interpretation & the Greeks as a Taylor Expansion

Each Greek is a local sensitivity: $\Delta = \partial V/\partial S$ (exposure to spot), $\Gamma = \partial^2 V/\partial S^2$ (curvature), Vega $= \partial V/\partial \sigma$ (volatility exposure), $\Theta = \partial V/\partial t$ (time decay), $\rho = \partial V/\partial r$ (rate exposure). A first-order approximation for small market moves is $\Delta V \approx \Delta\,\Delta S + \text{Vega}\,\Delta\sigma + \Theta\,\Delta t + \rho\,\Delta r$. Adding the second-order term in S gives the full local Taylor expansion underlying Greek-based P&L attribution:

$$\boxed{\Delta V \approx \Delta\,\Delta S + \tfrac{1}{2}\Gamma(\Delta S)^2 + \text{Vega}\,\Delta\sigma + \Theta\,\Delta t + \rho\,\Delta r}$$

Theta Revisited: Calendar Time t vs. Time-to-Maturity T

A subtlety: option pricing formulas are written in terms of T (time *to* maturity), but Theta is a derivative with respect to calendar time t. Since $T = T_{\text{maturity}} - t$:

$$t \uparrow \implies T \downarrow, \qquad \frac{\partial T}{\partial t} = -1.$$

By the chain rule, $\dfrac{\partial V}{\partial t} = \dfrac{\partial V}{\partial T}\dfrac{\partial T}{\partial t} = -\dfrac{\partial V}{\partial T}$.

Why this matters

Differentiating the pricing formula with respect to T (the natural variable in the formula) and then negating gives exactly the conventional, calendar-time Theta – this is why every Theta formula above carries a leading minus sign on its decay term.

Setup: Why the MGF?

Let $X \sim \mathcal{N}(\mu, \sigma^2)$, so $\mathbb{E}[X] = \mu$ and $\mathrm{Var}(X) = \sigma^2$. We want a general tool to compute $\mathbb{E}[e^{tX}]$ for any t – this is the **moment-generating function** (MGF):

$$M_X(t) = \mathbb{E}\left[e^{tX}\right].$$

Why it is called that

Differentiating M_X with respect to t and evaluating at $t = 0$ recovers the raw moments of X – we verify this first, then compute $M_X(t)$ explicitly for the normal distribution.

Recovering Moments: First and Second Derivatives

Differentiating under the expectation (valid since the normal MGF is finite for every t):

$$M_X'(t) = \frac{d}{dt}\mathbb{E}[e^{tX}] = \mathbb{E}\left[\frac{d}{dt}e^{tX}\right] = \mathbb{E}\left[Xe^{tX}\right] \quad \Longrightarrow \quad \boxed{M_X'(0) = \mathbb{E}[X]}$$

(using $e^{0X} = 1$). Differentiating once more:

$$M_X''(t) = \mathbb{E}\left[\frac{d}{dt}\left(Xe^{tX}\right)\right] = \mathbb{E}\left[X^2e^{tX}\right] \quad \Longrightarrow \quad \boxed{M_X''(0) = \mathbb{E}[X^2]}$$

Recovering Moments: General Pattern

Differentiating n times with respect to t:

$$\boxed{M_X^{(n)}(t) = \mathbb{E}\left[X^n e^{tX}\right]} \qquad \Longrightarrow \qquad \boxed{M_X^{(n)}(0) = \mathbb{E}[X^n]}$$

$$\boxed{\begin{aligned} &M_X(0) = 1, && M_X'(0) = \mathbb{E}[X], \\ &M_X''(0) = \mathbb{E}[X^2], && M_X'''(0) = \mathbb{E}[X^3], \ \ldots \end{aligned}}$$

The mechanism: $e^{tX} \xrightarrow{d/dt} Xe^{tX} \xrightarrow{d/dt} X^2 e^{tX} \xrightarrow{d/dt} \cdots$, then setting $t = 0$ makes $e^{0X} = 1$ at each step. For $X \sim \mathcal{N}(\mu, \sigma^2)$, we will show $\boxed{M_X(t) = e^{\mu t + \frac{1}{2}\sigma^2 t^2}}$.

Setting Up the Integral

By definition $M_X(t) = \mathbb{E}[e^{tX}] = \displaystyle\int_{-\infty}^{+\infty} e^{tx} f_X(x)\, dx$, with

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Combining the two exponentials:

$$M_X(t) = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{+\infty} \exp\left(tx - \frac{(x-\mu)^2}{2\sigma^2}\right) dx.$$

The key step is to simplify the exponent $tx - \dfrac{(x-\mu)^2}{2\sigma^2}$ by **completing the square**.

Completing the Square (1/2)

Expand $(x-\mu)^2 = x^2 - 2\mu x + \mu^2$, so

$$tx - \frac{x^2 - 2\mu x + \mu^2}{2\sigma^2} = -\frac{x^2 - 2(\mu + \sigma^2 t)x + \mu^2}{2\sigma^2}.$$

Using $x^2 - 2ax = (x-a)^2 - a^2$ with $a = \mu + \sigma^2 t$:

$$x^2 - 2(\mu + \sigma^2 t)x = \left(x - \mu - \sigma^2 t\right)^2 - (\mu + \sigma^2 t)^2.$$

Therefore

$$tx - \frac{(x-\mu)^2}{2\sigma^2} = -\frac{(x-\mu-\sigma^2 t)^2}{2\sigma^2} + \frac{(\mu+\sigma^2 t)^2 - \mu^2}{2\sigma^2}.$$

Completing the Square (2/2)

Expanding $(\mu + \sigma^2 t)^2 = \mu^2 + 2\mu\sigma^2 t + \sigma^4 t^2$, so $(\mu + \sigma^2 t)^2 - \mu^2 = 2\mu\sigma^2 t + \sigma^4 t^2$. Dividing by $2\sigma^2$:

$$\frac{2\mu\sigma^2 t + \sigma^4 t^2}{2\sigma^2} = \mu t + \tfrac{1}{2}\sigma^2 t^2.$$

Result

$$\boxed{tx - \frac{(x-\mu)^2}{2\sigma^2} = -\frac{(x-\mu-\sigma^2 t)^2}{2\sigma^2} + \mu t + \tfrac{1}{2}\sigma^2 t^2}$$

Returning to the Integral

Substituting back, the terms $\mu t + \frac{1}{2}\sigma^2 t^2$ do not depend on x and factor out of the integral:

$$M_X(t) = e^{\mu t + \frac{1}{2}\sigma^2 t^2} \cdot \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{+\infty} \exp\left(-\frac{(x - \mu - \sigma^2 t)^2}{2\sigma^2}\right) dx.$$

The remaining integrand is itself a normal density (mean $\mu + \sigma^2 t$, variance σ^2), which integrates to 1 over its support. Hence

$$\boxed{M_X(t) = e^{\mu t + \frac{1}{2}\sigma^2 t^2}}$$

Obtaining $\mathbb{E}[e^X]$

We want $\mathbb{E}[e^X]$. Since $e^{tX}\big|_{t=1} = e^X$, this is simply the MGF evaluated at $t = 1$:

$$\mathbb{E}[e^X] = M_X(1) = e^{\mu(1)+\frac{1}{2}\sigma^2(1)^2}.$$

Result

$$\boxed{\mathbb{E}[e^X] = e^{\mu+\frac{1}{2}\sigma^2}}$$

Application: Expected Gross and Net Return

Let the annual log return be $X \sim \mathcal{N}(0, 0.2^2)$, so $\mu = 0$, $\sigma = 0.2$, and $\mathbb{E}[X] = 0$. The gross simple return is $S_T/S_0 = e^X$, so

$$\mathbb{E}\left[\frac{S_T}{S_0}\right] = \mathbb{E}[e^X] = e^{\frac{1}{2}(0.2)^2} = e^{0.02} \approx \boxed{1.0202}.$$

The net simple return is $R = e^X - 1$, so

$$\boxed{\mathbb{E}[R] = e^{\mu + \frac{1}{2}\sigma^2} - 1} \qquad \Longrightarrow \qquad \mathbb{E}[R] = e^{0.02} - 1 \approx \boxed{2.02\%}$$

Key Takeaway

$$\boxed{M_X(t) = \mathbb{E}[e^{tX}] = e^{\mu t + \frac{1}{2}\sigma^2 t^2}} \quad \Longrightarrow \quad \boxed{\mathbb{E}[e^X] = M_X(1) = e^{\mu + \frac{1}{2}\sigma^2}}$$

If X is a normally distributed log return,

$$\boxed{\mathbb{E}[R_{\text{simple}}] = e^{\mu + \frac{1}{2}\sigma^2} - 1}$$

and in particular, with $\mu = 0$, $\sigma = 20\%$: $\mathbb{E}[R_{\text{simple}}] = e^{0.2^2/2} - 1 \approx 2.02\%$.

The Monte Carlo Idea

We simulate many possible future stock-price paths by drawing random daily returns from a normal distribution:

$$\boxed{S_0 \longrightarrow S_1 \longrightarrow S_2 \longrightarrow \cdots \longrightarrow S_N}$$

where each transition depends on a newly drawn random return. Repeating M times yields

$$S_N^{(1)}, S_N^{(2)}, \ldots, S_N^{(M)},$$

an empirical approximation of the distribution of the future price. **Model inputs:** S_0 (initial price), σ (annualized volatility), N (trading days simulated), M (number of simulations) – e.g. $S_0 = 100$, $\sigma = 20\%$, $N = 252$, $M = 10{,}000$.

From Annual to Daily Volatility

Suppose daily returns are i.i.d. with $\mathrm{Var}(R_t) = \sigma_d^2$. Under the additive approximation $R_{\text{year}} = \sum_{t=1}^{252} R_t$, independence gives

$$\mathrm{Var}\left(\sum_{t=1}^{252} R_t\right) = \sum_{t=1}^{252} \mathrm{Var}(R_t) \quad \Longrightarrow \quad \sigma^2 = 252\,\sigma_d^2.$$

Taking square roots – the **square-root-of-time rule**:

$$\boxed{\sigma_d = \frac{\sigma}{\sqrt{252}}}$$

For $\sigma = 20\%$: $\sigma_d = 0.20/\sqrt{252} \approx \boxed{1.26\%}$.

Generating Random Daily Returns

Each trading day, a new shock is drawn:

$$\boxed{R_t \sim \mathcal{N}(0, \sigma_d^2)} \qquad \text{equivalently} \qquad R_t = \sigma_d Z_t, \quad Z_t \sim \mathcal{N}(0,1),$$

with $\mathbb{E}[R_t] = 0$, $\mathrm{Var}(R_t) = \sigma_d^2$, and shocks independent across time: $Z_1, \ldots, Z_N \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$.

Discrete Price Update

With a **simple return** interpretation, the price updates as

$$\boxed{S_{t+1} = S_t(1 + R_{t+1})}, \qquad R_{t+1} \sim \mathcal{N}(0, \sigma_d^2).$$

After one day $S_1 = S_0(1 + R_1)$; after two days $S_2 = S_0(1 + R_1)(1 + R_2)$; after N days, returns compound **multiplicatively**:

$$\boxed{S_N = S_0 \prod_{t=1}^{N} (1 + R_t)}$$

Expected Price in the Discrete Simulation

Since $\mathbb{E}[R_t] = 0$, we have $\mathbb{E}[1 + R_t] = 1$. Assuming independence,

$$\mathbb{E}[S_N] = S_0 \, \mathbb{E}\left[\prod_{t=1}^{N}(1 + R_t)\right] = S_0 \prod_{t=1}^{N} \mathbb{E}[1 + R_t] = S_0 \prod_{t=1}^{N} 1.$$

Result

$$\boxed{\mathbb{E}[S_N] = S_0}$$

With $S_0 = 100$, the average terminal price across many discrete simulations should be close to 100.

Continuous Price Update

With a **log return** interpretation, the price updates as

$$\boxed{S_{t+1} = S_t \, e^{R_{t+1}}} \qquad \Longleftrightarrow \qquad \boxed{R_{t+1} = \ln\left(\frac{S_{t+1}}{S_t}\right)}.$$

Indeed, $S_{t+1} = S_t e^{R_{t+1}} \Rightarrow S_{t+1}/S_t = e^{R_{t+1}} \Rightarrow \ln(S_{t+1}/S_t) = R_{t+1}$.

Compounding Over Multiple Days

After N days, $S_N = S_0\, e^{R_1} e^{R_2} \cdots e^{R_N}$. Using $e^a e^b = e^{a+b}$:

$$\boxed{S_N = S_0 \exp\left(\sum_{t=1}^{N} R_t\right)}$$

Since $R_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma_d^2)$, the sum is normal: $\sum_{t=1}^{N} R_t \sim \mathcal{N}(0, N\sigma_d^2)$. For $N = 252$ and $\sigma_d^2 = \sigma^2/252$: $N\sigma_d^2 = \sigma^2$, so

$$\boxed{\ln\left(\frac{S_{252}}{S_0}\right) \sim \mathcal{N}(0, \sigma^2)} \qquad \Longrightarrow \qquad \boxed{S_{252} = S_0 e^X,\ X \sim \mathcal{N}(0, \sigma^2)}$$

The terminal price is therefore **lognormal**.

Expected Price in the Continuous Simulation

Recall $\mathbb{E}[e^X] = e^{\mu + \frac{1}{2}v}$ for $X \sim \mathcal{N}(\mu, v)$. Here $X = \sum_{t=1}^{N} R_t$ has $\mu = 0$, $v = N\sigma_d^2$, so $\mathbb{E}[S_N] = S_0\,\mathbb{E}[e^X]$ gives

$$\boxed{\mathbb{E}[S_N] = S_0\, e^{\frac{1}{2}N\sigma_d^2}} \quad \overset{N=252}{\Longrightarrow} \quad \boxed{\mathbb{E}[S_{252}] = S_0\, e^{\frac{1}{2}\sigma^2}}$$

With $S_0 = 100, \sigma = 0.20$: $\mathbb{E}[S_{252}] = 100\, e^{0.02} \approx \boxed{102.02}$ – **strictly above** S_0, unlike the discrete case.

Jensen's Inequality at Work

Discrete: $\mathbb{E}[S_{t+1}/S_t] = \mathbb{E}[1 + R_{t+1}] = 1$ exactly (since $\mathbb{E}[R_t] = 0$). Continuous: although $\mathbb{E}[R_t] = 0$ still, $\mathbb{E}[e^{R_t}] = e^{\frac{1}{2}\sigma_d^2} > 1$ (convexity of exp).

Simulation	**Price update**	$\mathbb{E}[S_{252}]$
Discrete	$S_{t+1} = S_t(1 + R_{t+1})$	S_0
Continuous	$S_{t+1} = S_t e^{R_{t+1}}$	$S_0 e^{\sigma^2/2}$

No free lunch

A higher arithmetic mean of simulated *prices* does not mean systematic profit: it reflects the asymmetry of the lognormal distribution (mean > median), i.e. **volatility drag / geometric decay** – the typical (median) path stays at S_0 when the drift is zero.

The Monte Carlo Estimator

Repeating the simulation M times produces the sample $S_N^{(1)}, \ldots, S_N^{(M)}$. The theoretical expectation $\mathbb{E}[S_N]$ is estimated by the sample average:

$$\widehat{\mathbb{E}[S_N]} = \frac{1}{M} \sum_{m=1}^{M} S_N^{(m)}$$

Law of Large Numbers

$$\frac{1}{M} \sum_{m=1}^{M} S_N^{(m)} \longrightarrow \mathbb{E}[S_N] \qquad \text{as } M \to \infty$$

More simulations make the Monte Carlo estimate increasingly precise.

The Empirical Distribution of Terminal Prices

A histogram of $S_N^{(1)}, \ldots, S_N^{(M)}$ is an empirical approximation of the **entire distribution** of S_N, not just its mean:

$$\boxed{\{S_N^{(m)}\}_{m=1}^{M} \longrightarrow \text{distribution of } S_N}$$

From this empirical distribution one can estimate $\mathbb{E}[S_N]$, $\mathrm{Var}(S_N)$, $\mathbb{P}(S_N < K)$, or any quantile of S_N – this is precisely the Monte Carlo approach to pricing and risk measurement used later in the course (e.g. Monte Carlo VaR in TD4).

Summary of the Monte Carlo Algorithm

For each simulation $m = 1, \ldots, M$: set $S_0^{(m)} = S_0$; for each day $t = 0, \ldots, N-1$, draw $Z_{t+1}^{(m)} \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$ and set $R_{t+1}^{(m)} = \dfrac{\sigma}{\sqrt{252}} Z_{t+1}^{(m)}$; update

$$\boxed{S_{t+1}^{(m)} = S_t^{(m)}(1 + R_{t+1}^{(m)})} \quad \text{(discrete)} \qquad \text{or} \qquad \boxed{S_{t+1}^{(m)} = S_t^{(m)} e^{R_{t+1}^{(m)}}} \quad \text{(continuous)}.$$

After M runs, $\widehat{\mathbb{E}[S_N]} = \frac{1}{M} \sum_m S_N^{(m)}$. For $S_0 = 100, \sigma = 20\%, N = 252$:

$$\boxed{\text{Discrete: } \mathbb{E}[S_{252}] = 100 \qquad \text{Continuous: } \mathbb{E}[S_{252}] = 100 e^{0.2^2/2} \approx 102.02}$$

String Checks: Alphabetic and Case

Two elementary character checks matter for any cyclic-shift cipher: `isalpha()` answers "*is this character a letter?*" (so non-letters are left unchanged by the cipher), and `isupper()` answers "*is this character uppercase?*" (so the cipher can preserve the original capitalization, choosing a base code point of 65 for *A* or 97 for *a* accordingly).

Subtlety

`isalpha()` recognizes any Unicode letter (e.g. accented characters), but a 26-letter cyclic-shift cipher is arithmetic on the Latin alphabet $A, \ldots, Z$ specifically – accented letters need a stricter check if they must be handled correctly.

The Modulo Operator

The operator a mod b returns the remainder of the Euclidean division of a by b: if $a = qb + r$ with q the quotient, then $\boxed{a \bmod b = r}$. Example: $7 = 2 \times 3 + 1 \Rightarrow \boxed{7 \bmod 3 = 1}$. More examples modulo 4:

$$5 \bmod 4 = 1, \quad 6 \bmod 4 = 2, \quad 7 \bmod 4 = 3, \quad 8 \bmod 4 = 0, \quad 9 \bmod 4 = 1.$$

The values repeat periodically: $\boxed{0, 1, 2, 3, 0, 1, 2, 3, \ldots}$ – this periodicity is exactly what makes modulo useful for anything cyclic.

Using Modulo to Cycle Through a Key

With key `"clef"` ($m = 4$ letters), the key position at step i is $i \bmod 4$:

i	0	1	2	3	4	5	6	7	8
$i \bmod 4$	0	1	2	3	0	1	2	3	0
letter	c	l	e	f	c	l	e	f	c

$i \bmod m$ cycles through $0, 1, \ldots, m-1, 0, 1, \ldots$

The key therefore repeats: $c, l, e, f, c, l, e, f, \ldots$

Using Modulo to Wrap Around the Alphabet

Represent letters as $A \to 0, B \to 1, \ldots, Z \to 25$. Shift Y (code 24) by 5: $24 + 5 = 29$, which is outside $\{0, \ldots, 25\}$. Taking modulo 26: $29 \bmod 26 = 3 \to D$.

$$\boxed{Y \xrightarrow{+5} D}$$

Modulo 26 makes the alphabet **cyclic**: $\cdots \to X \to Y \to Z \to A \to B \to C \to \cdots$. The shift transformation is therefore, for letter code x_i and key-letter shift k_i (both in $\{0, \ldots, 25\}$):

$$\boxed{y_i = (x_i + k_i) \bmod 26}$$

Summary: Two Roles of Modulo in a Cyclic Cipher

$i \bmod m$	$\longrightarrow$	repeat the key cyclically
$(x_i + k_i) \bmod 26$	$\longrightarrow$	wrap around the alphabet

Check/operator	**Role**	**Example**
`.isalpha()`	is this a letter?	`"A"` $\rightarrow$ True
`.isupper()`	is it uppercase?	`"A"` $\rightarrow$ True
`a % b`	remainder of $a \div b$	$7 \bmod 3 = 1$